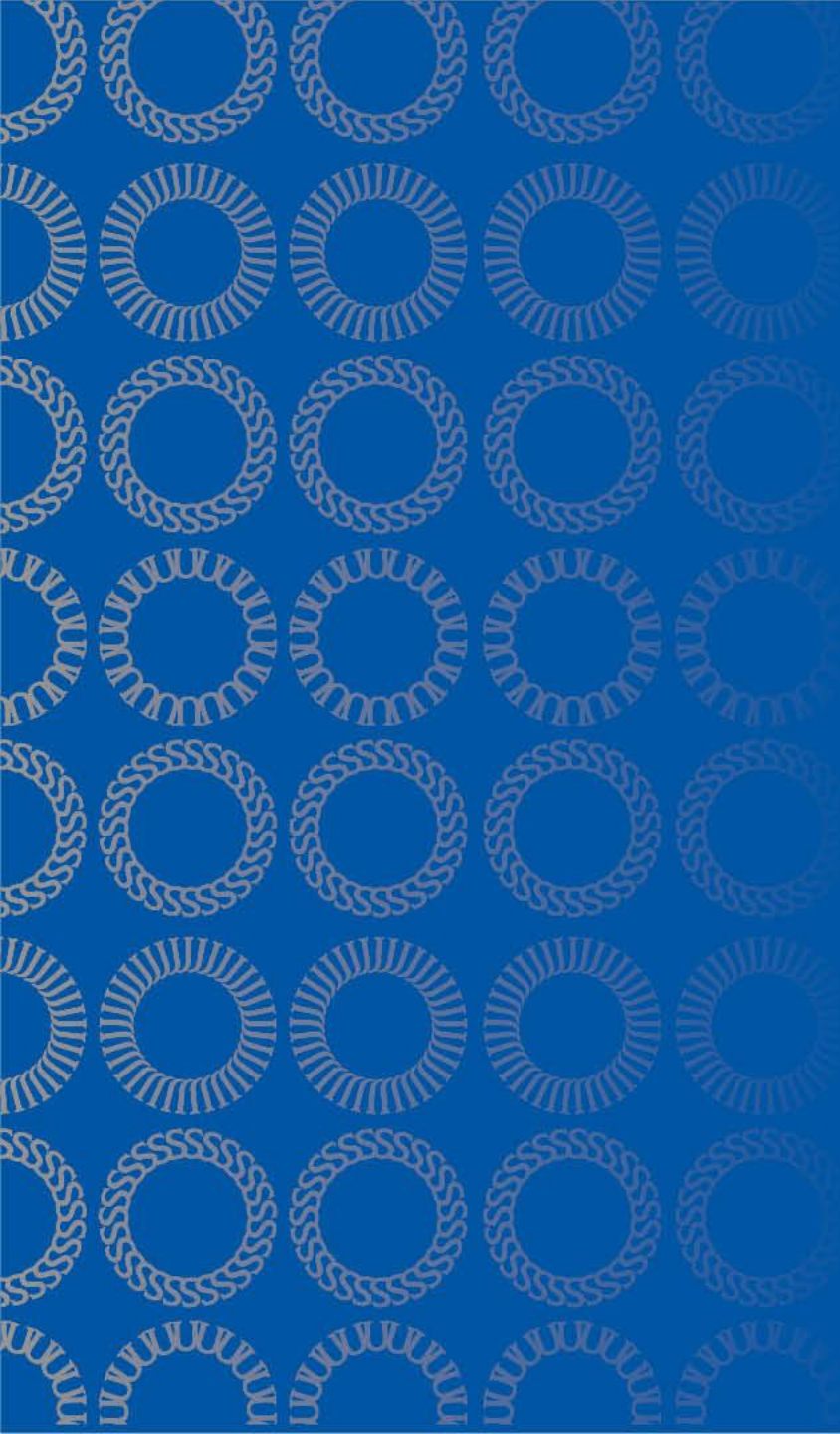




DATA 255 Deep Learning Technologies

Sept 18, 2025

Taehee Jeong, Ph.D.

Today's Topics

- Learning rate
- Optimizer

Batch & Stochastic gradient descent

**Batch
Gradient
descent**

$$\frac{\partial J}{\partial W} = -\frac{1}{m} \sum_{i=1}^m [(\hat{y}^i - y^i)x_j^i] \quad (\text{Sum over data points})$$

**Stochastic
Gradient
descent**

$$\frac{\partial J}{\partial W} = -(\hat{y}^i - y^i)x_j^i \quad (\text{Each time, pick different data point})$$

Mini-batch Gradient Descent

- Batch Gradient Descent

Computes the gradients based on full training set

Ex) Offline learning

- Stochastic Gradient Descent

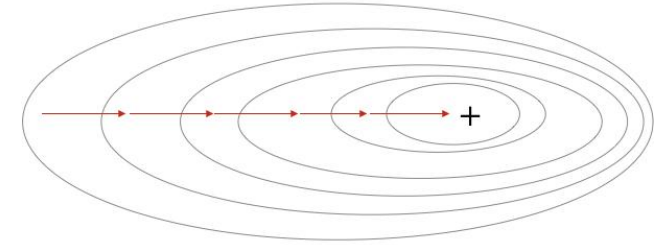
Computes just one instance

Ex) Online learning

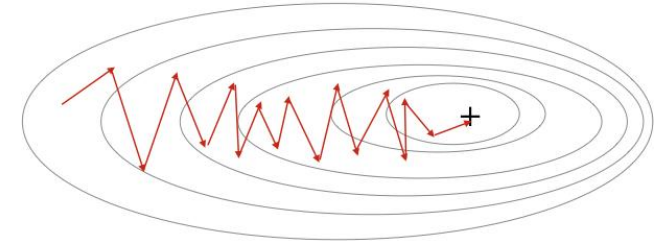
- Mini-batch Gradient Descent

Computes the gradients on small random sets of instances

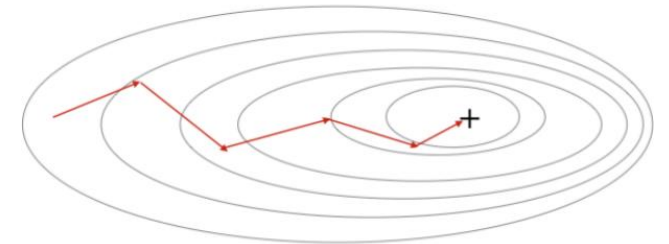
Gradient Descent



Stochastic Gradient Descent

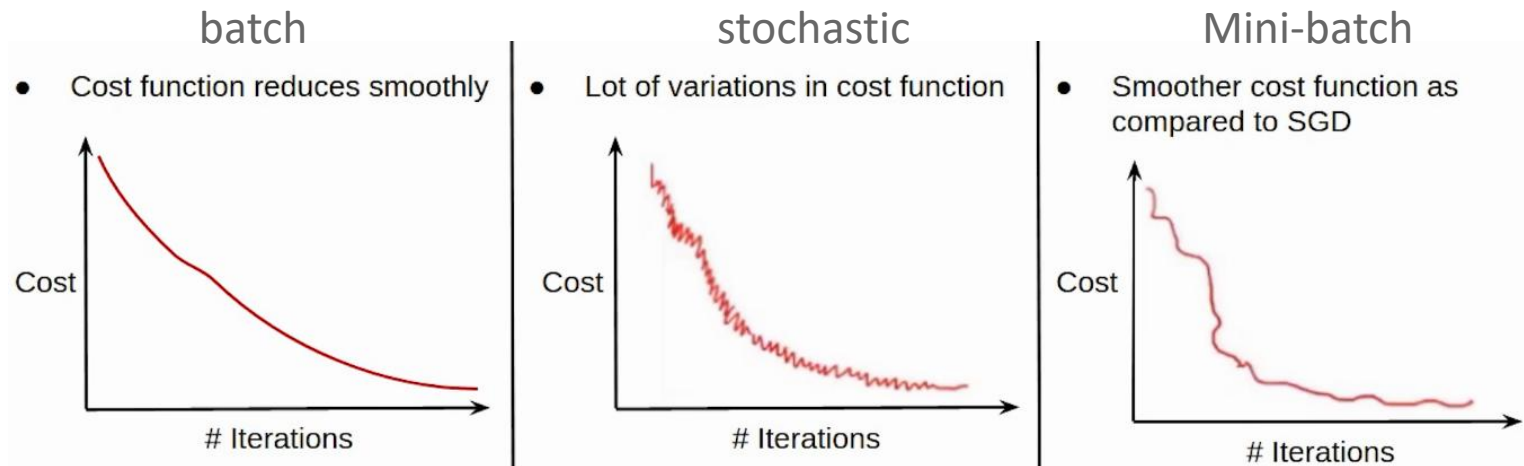


Mini-Batch Gradient Descent

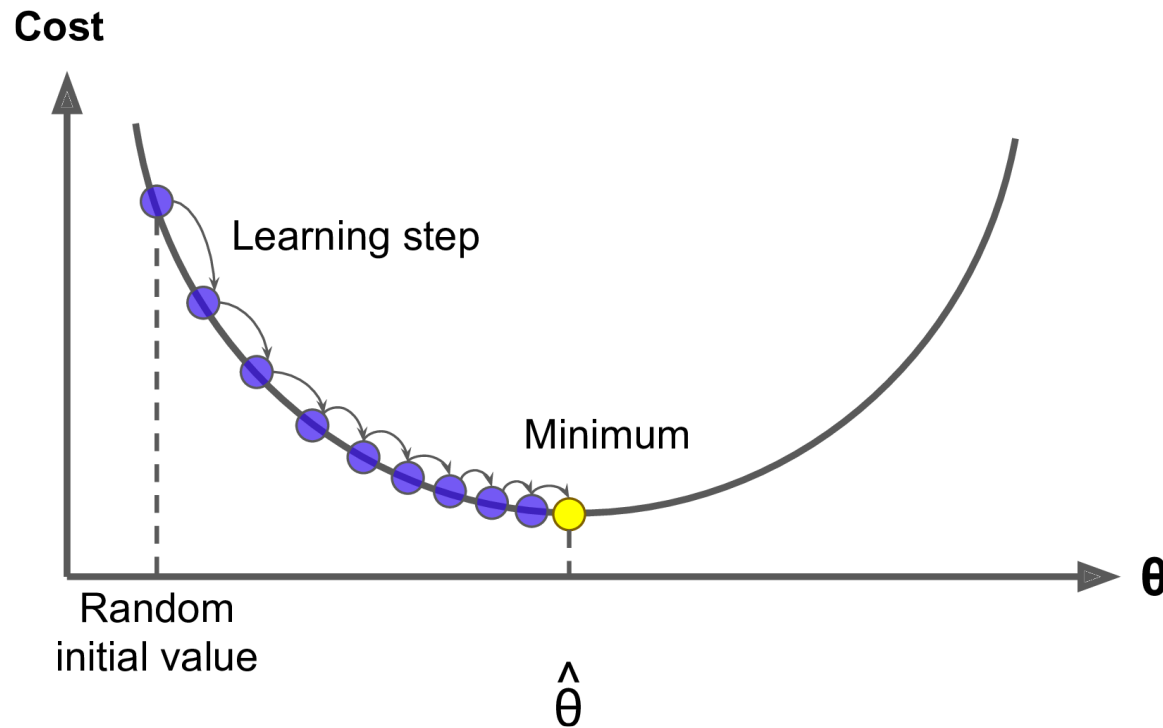


Batch & Stochastic gradient descent

- "batch gradient descent" calculates the gradient using the entire training dataset for each parameter update
- "stochastic gradient descent" updates parameters based on the gradient calculated from just one randomly selected data point at a time, making it significantly faster for large datasets but potentially less stable due to the noisy updates



Gradient Descent



$$W_j^t := W_j^{t-1} - \alpha \frac{\partial J}{\partial W}$$

When Learning rate is too high

If learning rate is large, parameters decay too aggressively and the parameters are unable to reach the local minimum.

$$W_j^t := W_j^{t-1} - \alpha \frac{\partial J}{\partial W}$$

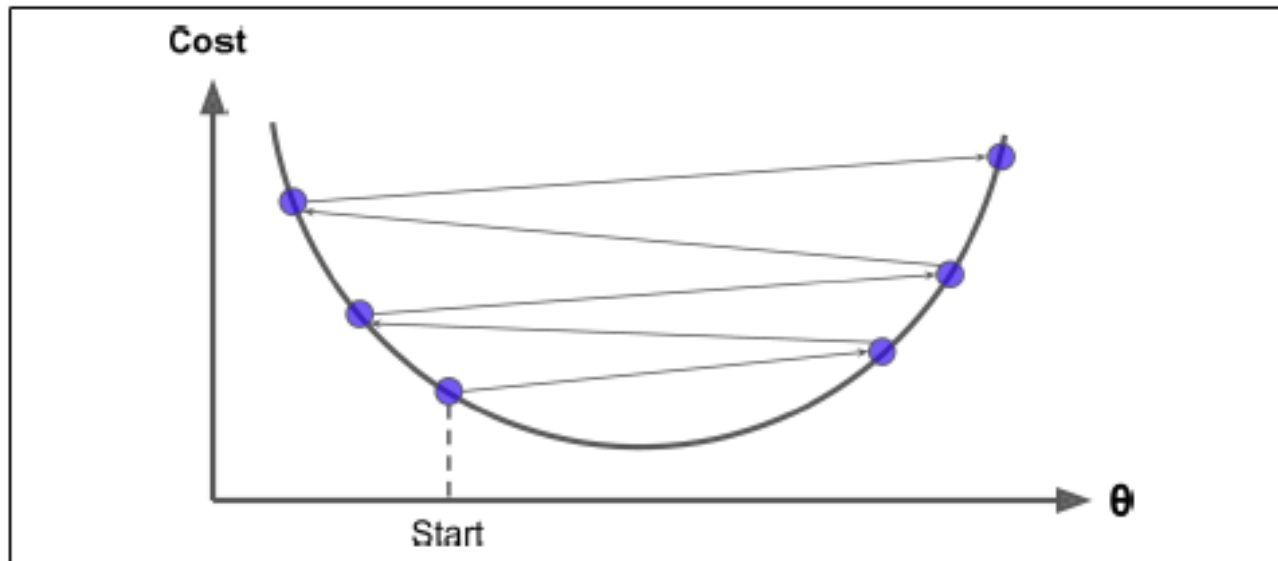
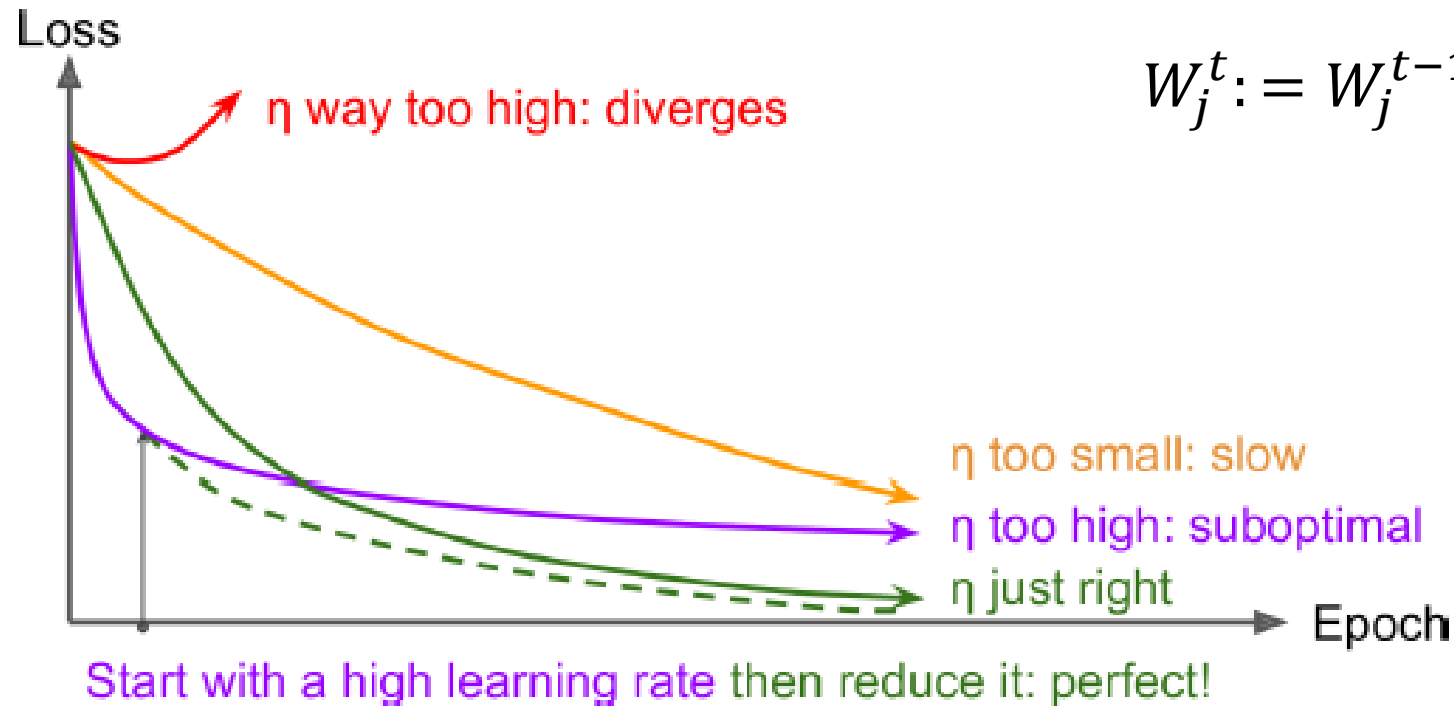


Figure 4-5. Learning rate too large

Loss vs. Learning rate (or step size)

1 epoch : 1 pass through data set, iteration number



$$W_j^t := W_j^{t-1} - \alpha \frac{\partial J}{\partial W}$$

Learning rate decay

Slowing decreasing α

- $1/t$ decay
- Exponentially decay
- Step decay: discrete staircase
- Discrete staircase

1/t decay

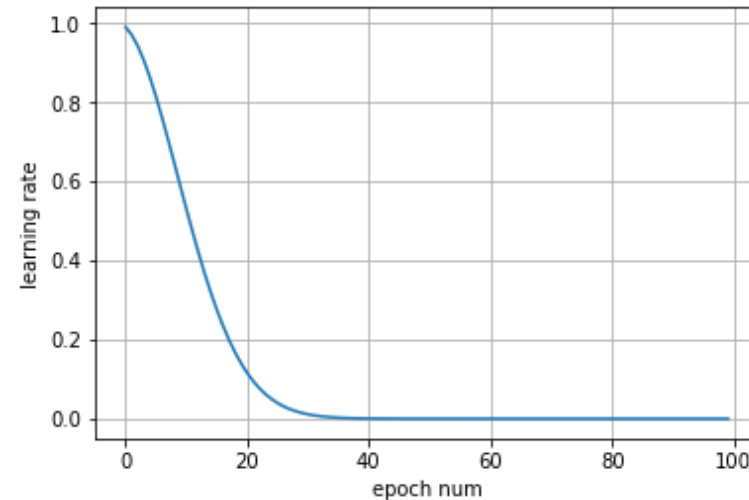
$$\alpha = \frac{\alpha_0}{1 + \text{decay_rate} \times \text{epoch_num}} = \frac{\alpha_0}{1 + k \cdot t}$$

$$\alpha_0 = 1$$

decay_rate(k) = 0.1



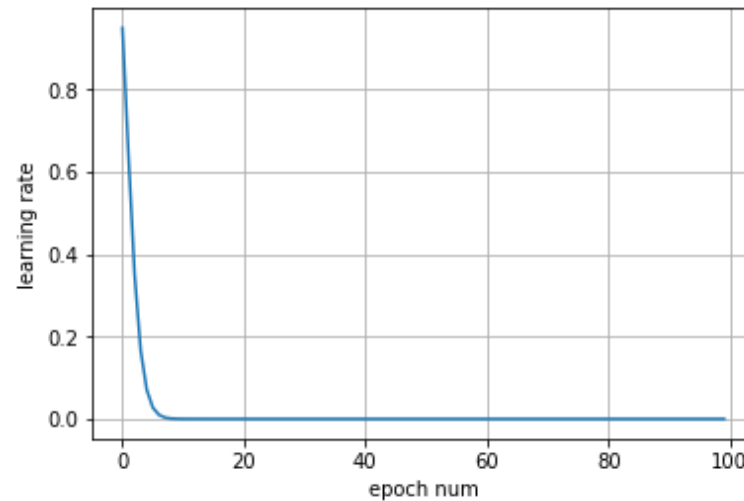
decay_rate(k) = 0.01



1/t decay

$$\alpha = \frac{k \times \alpha_0}{\sqrt{\text{epoch_num}}} = \frac{k \times \alpha_0}{\sqrt{t}}$$

$$\alpha_0 = 1$$
$$k = 0.95$$

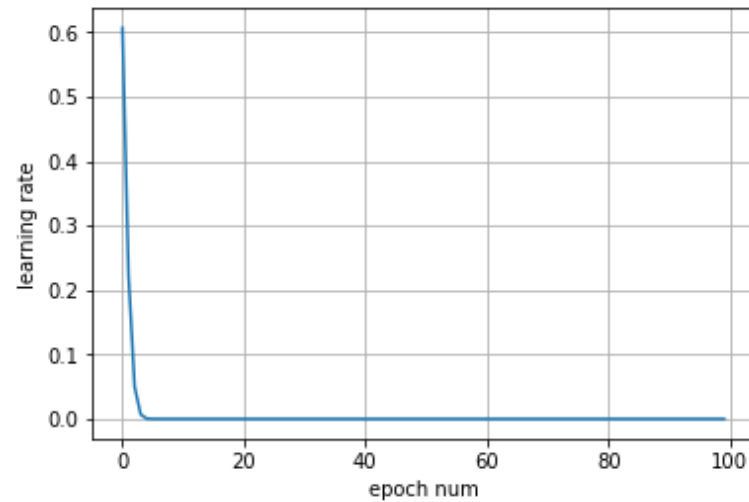


Exponential decay

$$\alpha = \alpha_0 \cdot \exp(-kt)$$

$$\alpha_0 = 1$$

$$k = 0.5$$

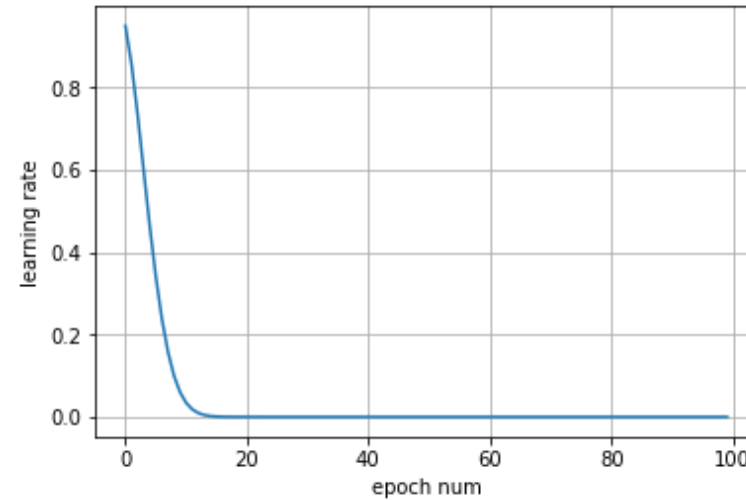


Exponential decay

$$\alpha = k^{epoch_num} \times \alpha_0 = k^t \times \alpha_0$$

$$\alpha_0 = 1$$

$$k = 0.95$$



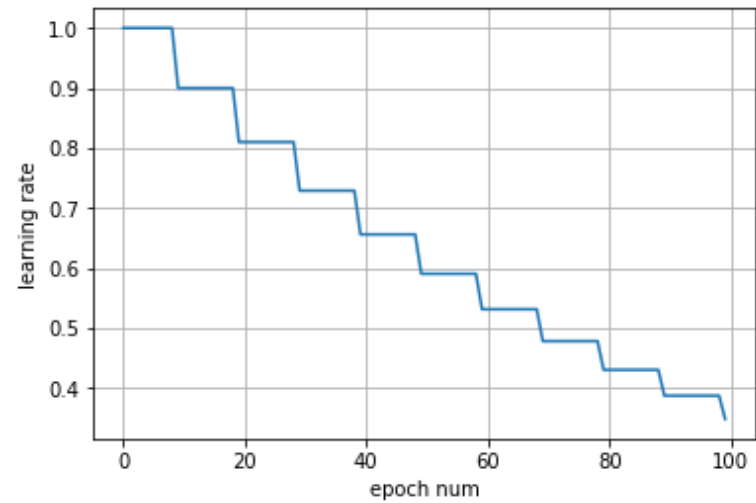
Discrete staircase

$\alpha = k \times \alpha$ with every few epochs

$$\alpha_0 = 1$$

$$k = 0.9$$

every 10 epochs



Learning rate decay

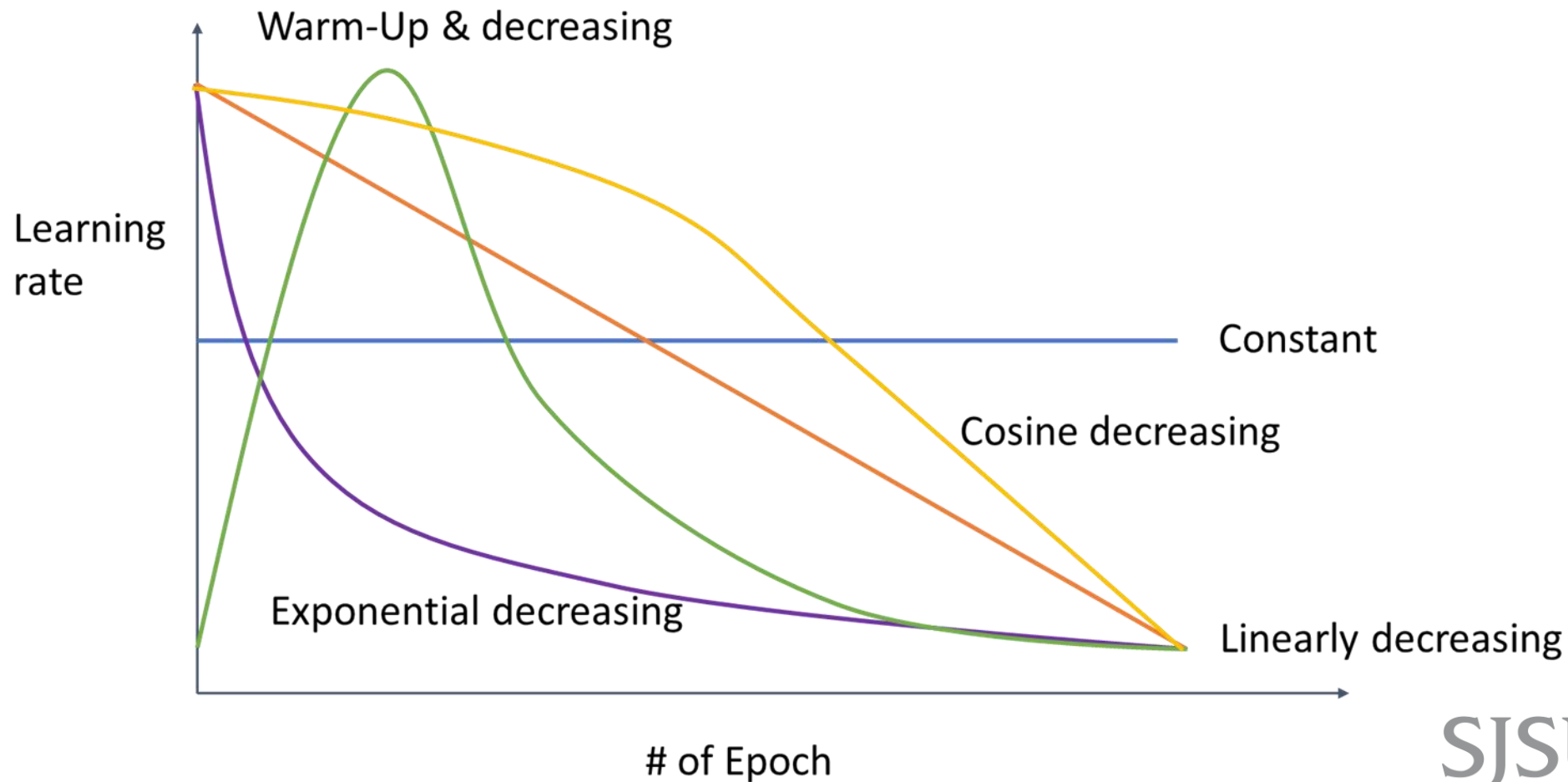
Slowing decreasing α

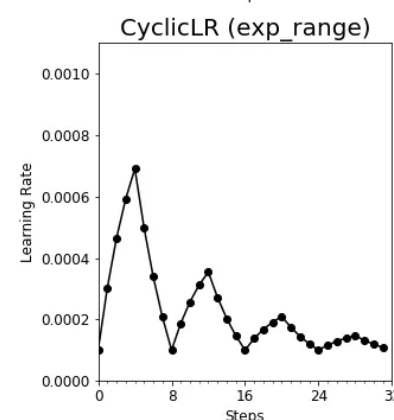
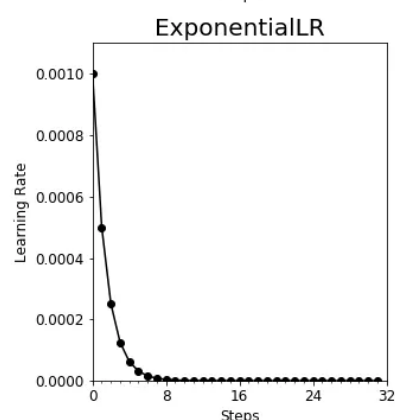
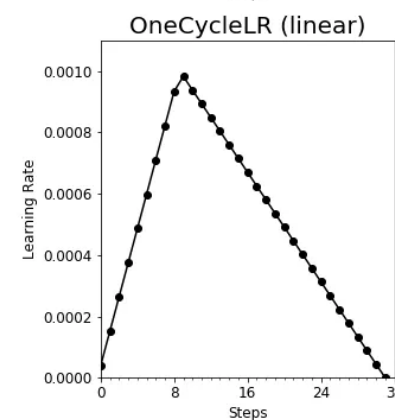
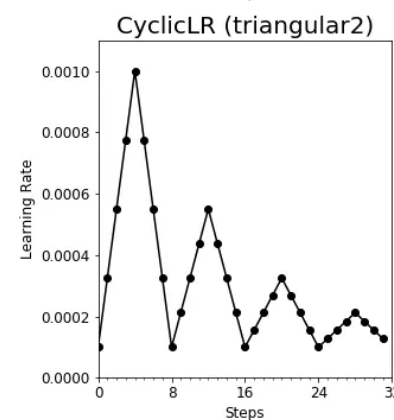
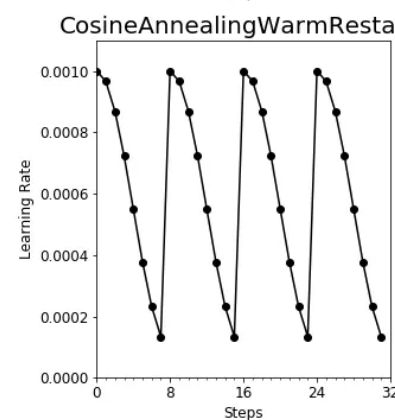
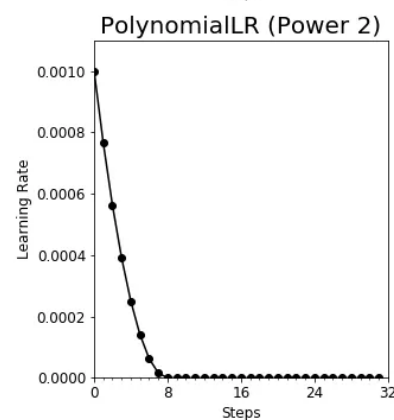
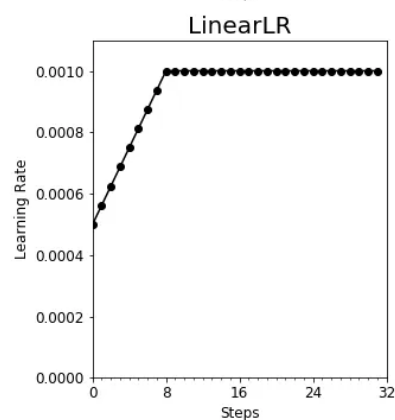
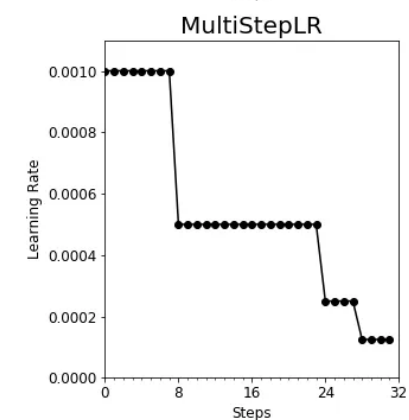
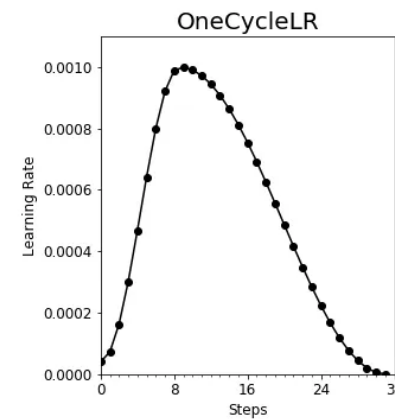
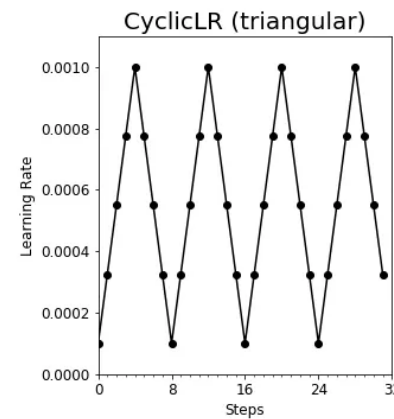
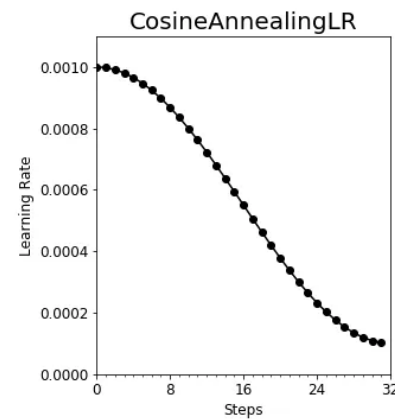
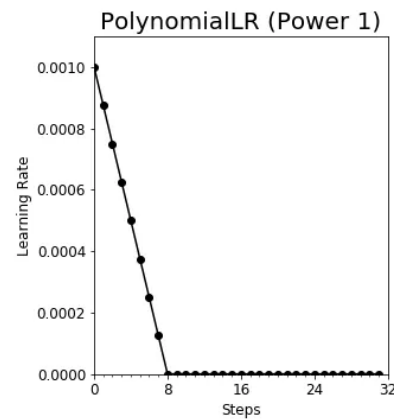
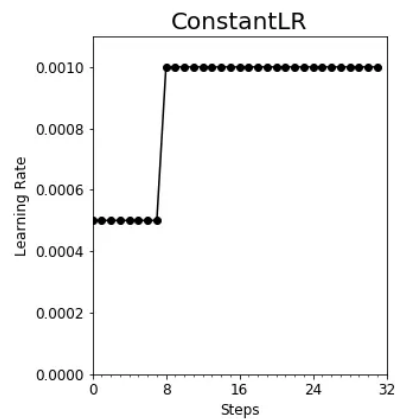
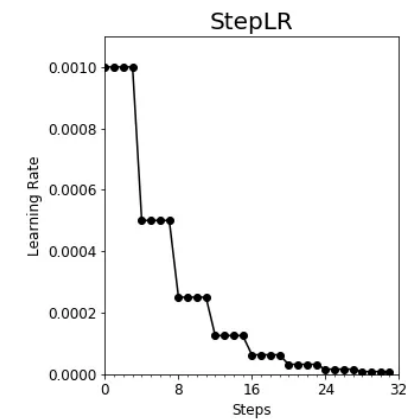
- 1/t decay: $\alpha = \frac{\alpha_0}{1 + kt}$ | $\alpha = \frac{k \times \alpha_0}{\sqrt{t}}$
- Exponentially decay $\alpha = \alpha_0 \cdot \exp(-kt)$ | $\alpha = k^t \times \alpha_0$
- Step decay: discrete staircase
 $\alpha = k \times \alpha$ with every t

Choosing the right step size

- Using a fixed step size
- Gradually shrinking the step size over time
- Warm-up & shrinking

Learning rate over Epoch

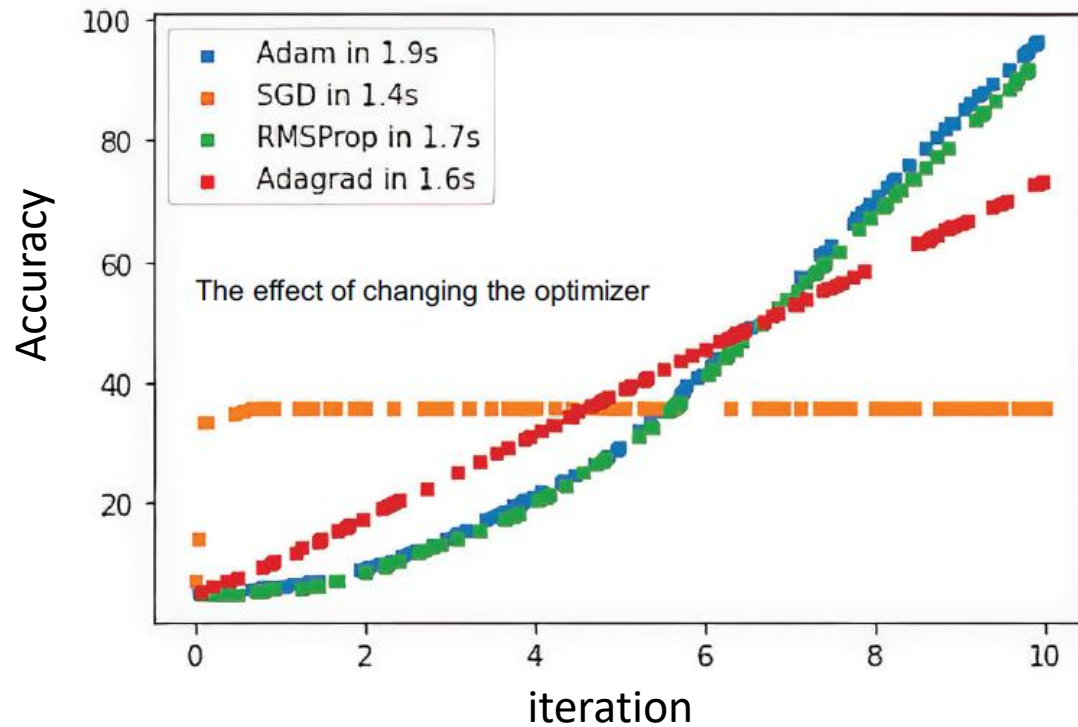




Parameter update (Faster optimization)

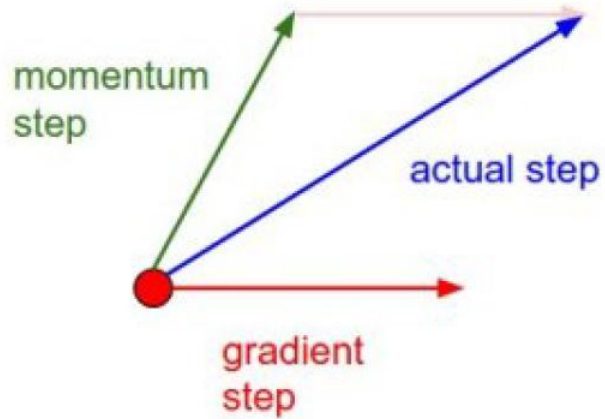
- Momentum-based Gradient optimizer
- Nesterov momentum → not cover
- Exponentially Weighted Averages
- Adagrad
- Rmsprop
- Adam
- Second order method: based on Newton's method, L-BFGS → not cover

Comparison of optimizer functions



Momentum update

Gradient descent update
 $x := x - \text{learning rate} * dx$



Momentum update

$$v = \beta * v - \text{learning rate} * dx$$

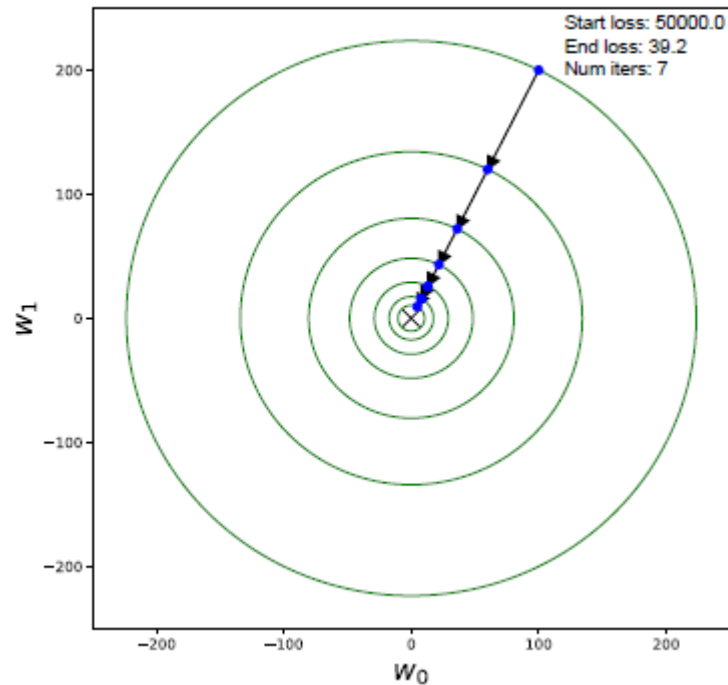
$$x := x + v$$

v : velocity

β : momentum

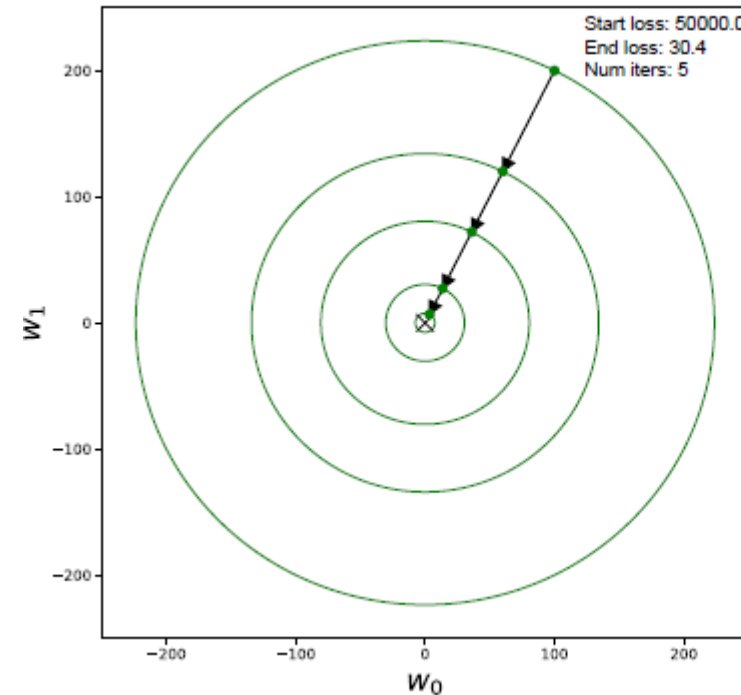
Gradient descent with/wo Momentum

Gradient descent without momentum for loss $L(w_0, w_1) = w_0^2 + w_1^2 = k$



(a) A trajectory through constant loss contours for gradient descent *without* momentum

Gradient descent with momentum for loss $L(w_0, w_1) = w_0^2 + w_1^2 = k$



(b) A trajectory through constant loss contours for gradient descent *with* momentum

Adagrad

Adaptive learning rate method

$$v := v + dx^{**2}$$

$$x := x - \text{learning_rate} * dx / (\text{sqrt}(v) + \text{eps})$$

v : sum of squared gradients

Normalized the parameter update step, element-wise.

Weights with high gradients will have effective learning rate reduces,

Weights with low gradients will have effective learning rate increases.

Gradient descent with momentum

$$V_{dW} = 0, V_{db} = 0$$

For every iteration t:

Compute V_{dW} , V_{db} using dW and db

$$V_{dW} := \beta_1 \cdot V_{dW} + (1 - \beta_1) \cdot dW$$

$$V_{db} := \beta_1 \cdot V_{db} + (1 - \beta_1) \cdot db$$

$$W := W - \alpha \cdot V_{dW}$$

$$b := b - \alpha \cdot V_{db}$$

RMS prop

Root Mean square prop

$$S_{dW} \neq 0, S_{db} \neq 0, \epsilon = 10^{-8}$$

For every iteration t:

Compute S_{dW} , S_{db} using dW and db

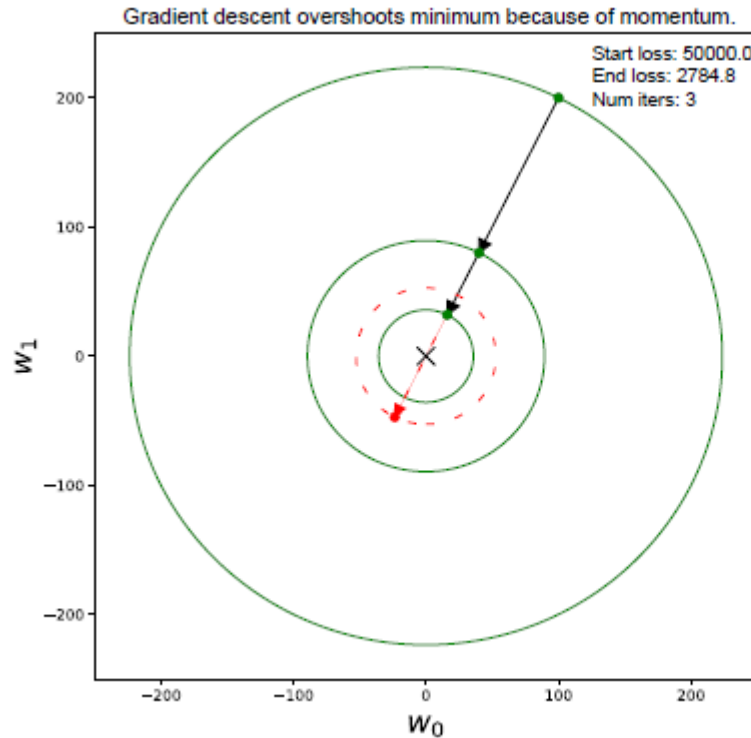
$$S_{dW} := \beta_2 \cdot S_{dW} + (1 - \beta_2) \cdot dW^2$$

$$S_{db} := \beta_2 \cdot S_{db} + (1 - \beta_2) \cdot db^2$$

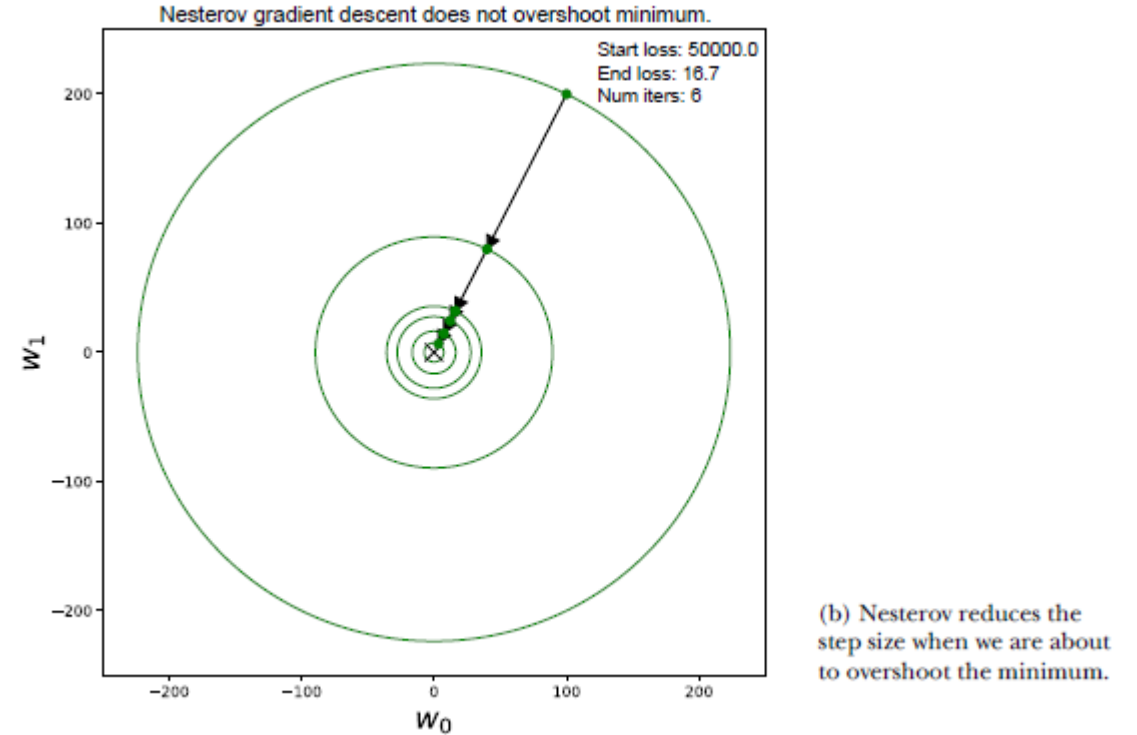
$$W := W - \alpha \cdot \frac{dW}{\sqrt{S_{dW} + \epsilon}}$$

$$b := b - \alpha \cdot \frac{db}{\sqrt{S_{db} + \epsilon}}$$

Nesterov accelerated gradient



(a) Momentum gradient descent overshooting the minimum. Loss decreases for a while and then increases (dotted circle and arrow).



(b) Nesterov reduces the step size when we are about to overshoot the minimum.

Adam Optimization

Adaption moment estimation

For every iteration t :
 $V_{dW} = 0, V_{db} = 0, S_{dW} = 0, S_{db} = 0, \epsilon = 10^{-8}$

Compute V_{dW} , V_{db} , S_{dW} , S_{db} using dW and db

$$V_{dW} := \beta_1 \cdot V_{dW} + (1 - \beta_1) \cdot dW$$

$$V_{db} := \beta_1 \cdot V_{db} + (1 - \beta_1) \cdot db$$

$$S_{dW} := \beta_2 \cdot S_{dW} + (1 - \beta_2) \cdot dW^2$$

$$S_{db} := \beta_2 \cdot S_{db} + (1 - \beta_2) \cdot db^2$$

$$W := W - \alpha \cdot V_{dW}$$

$$b := b - \alpha \cdot V_{db}$$

Adam Optimization

Bias correction

$$V_{dW}^{cor} = \frac{V_{dW}}{1 - \beta_1^t}$$

$$V_{db}^{cor} = \frac{V_{db}}{1 - \beta_1^t}$$

$$S_{dW}^{cor} = \frac{S_{dW}}{1 - \beta_2^t}$$

$$S_{db}^{cor} = \frac{S_{db}}{1 - \beta_2^t}$$

$$W := W - \alpha \cdot \frac{V_{dW}^{cor}}{\sqrt{S_{dW}^{cor} + \epsilon}}$$

$$b := b - \alpha \cdot \frac{V_{db}^{cor}}{\sqrt{S_{db}^{cor} + \epsilon}}$$

Adam optimization

Hyper parameters choices

α : need to be tune

β_1 : 0.9 (momentum)

β_2 : 0.999 (RMS prop)

ε : 10^{-8}

Summary

SGD optimizer

$$W_j^t := W_j^{t-1} - \alpha \frac{\partial J}{\partial W}$$

Momentum optimizer

$$m^t = \beta \cdot m^{t-1} - \alpha \frac{\partial J}{\partial W}$$

$$W_j^t := W_j^{t-1} + m^t$$

$$W_j^t := W_j^{t-1} + \beta \cdot m^{t-1} - \alpha \frac{\partial J}{\partial W}$$

Weighted Averages

$$m^t = \beta \cdot m^{t-1} + (1 - \beta) \frac{\partial J}{\partial W}$$

$$W_j^t := W_j^{t-1} - \alpha \cdot m^t$$

$$W_j^t := W_j^{t-1} - \alpha \cdot [\beta \cdot m^{t-1} + (1 - \beta) \frac{\partial J}{\partial W}]$$

Adagrad

$$v^t = \sum_{i=0}^{t-1} \left(\frac{\partial J}{\partial W^i} \right)^2$$

$$W_j^t := W_j^{t-1} - \frac{\alpha}{\sqrt{v^t + \epsilon}} \cdot \frac{\partial J}{\partial W}$$

RMSprop

$$v^t = \beta \cdot v^{t-1} + (1 - \beta) \cdot \left(\frac{\partial J}{\partial W} \right)^2$$

$$W_j^t := W_j^{t-1} - \frac{\alpha}{\sqrt{v^t + \epsilon}} \cdot \frac{\partial J}{\partial W}$$

Adam

$$m^t := \beta_1 \cdot m^{t-1} + (1 - \beta_1) \cdot \frac{\partial J}{\partial W}$$

$$v^t = \beta_2 \cdot v^{t-1} + (1 - \beta_2) \cdot \left(\frac{\partial J}{\partial W} \right)^2$$

$$W_j^t := W_j^{t-1} - \alpha \cdot \frac{\widehat{m}^t}{\sqrt{\widehat{v}^t + \epsilon}} \cdot \frac{\partial J}{\partial W}$$

$$\widehat{m}^t = \frac{m^t}{1 - \{\beta_1\}^t}$$

$$\widehat{v}^t = \frac{v^t}{1 - \{\beta_2\}^t}$$